

Automatic and Continuous Software Architecture Validation

Maayan Goldstein, Itai Segall
IBM Research—Haifa
Mount Carmel, Haifa 31905, Israel
{maayang, itais}@il.ibm.com

Abstract—Software systems tend to suffer from architectural problems as they are being developed. While modern software development methodologies such as Agile and Dev-Ops suggest different ways of assuring code quality, very little attention is paid to maintaining high quality of the architecture of the evolving systems. By detecting and alerting about violations of the intended software architecture, one can often avoid code-level bad smells such as spaghetti code.

Typically, if one wants to reason about the software architecture, the burden of first defining the intended architecture falls on the developer’s shoulders. This includes definition of valid and invalid dependencies between software components. However, the developers are seldom familiar with the entire software system, which makes this task difficult, time consuming and error-prone.

We propose and implement a solution for automatic detection of architectural violations in software artifacts. The solution, which utilizes a number of predefined and user-defined patterns, does not require prior knowledge of the system or its intended architecture. We propose to leverage this solution as part of the nightly build process used by development teams, thus achieving continuous automatic validation of the system’s software architecture. As we show in multiple open-source and proprietary cases, a small set of predefined patterns can detect architectural violations as they are introduced over the course of development, and also capture deterioration in existing architectural problems. By evaluating the tool on relatively large open-source projects, we also validate its scalability and practical applicability to large software systems.

I. INTRODUCTION

With the increasing shift to agile development processes and the recent rise of Dev-Ops, changes to software are becoming more and more frequent. In order to ensure a continuous level of high quality of the code, these development processes suggest using Model-Driven Development (MDD) [4] and Test-Driven Development (TDD) [26], and preach for continuous integration and testing [10]. These approaches are meant to ensure a steady state of conformance with functional and non-functional requirements of the running code.

However, the software architecture gets little to no attention in these processes. As the code evolves, the quality of the developed software systems may be compromised, as they tend to suffer from various architectural problems [8], [38], [42]. Rapid development and release cycles, such as those stemming from Agile and Dev-Ops processes, may make

the situation even worse with their focus on small changes between commits and releases, thus allowing less time for considering “the large picture” – in our case, the overall architecture of the software system.

Therefore – just as with the introduction of the Object Oriented paradigm, where code quality became an important aspect of the software development process [35] – we believe that software architecture quality should become an important aspect of these rapid development approaches. This belief is supported by extensive research in the areas of component interactions, architectural patterns, high level code abstractions, and conformance between code and design [1], [28], [35], [36].

Unfortunately, software architecture reviews are expensive [22] and tool support is very limited [33], which makes them impractical for rapid development cycles. Therefore, there is a great need to develop new tools and integrate them within development environments. To continue the analogy with code quality – just like developers expect nowadays a continuous feedback on code-level issues as soon as they are introduced, e.g. with tools like FindBugs for Eclipse [23], we envision an IDE that supplies continuous feedback on architecture-level issues.

This concern was addressed in [13] where the authors suggested specifying interfaces between software components and validating the developed code against these interfaces. Although this makes software architecture validation a part of the continuous build and test process, it puts a lot of burden of specifying the interfaces on the developer’s shoulders.

We suggest an automated approach for detecting architectural problems. Our solution provides insights on systems of multi-million lines of code, and requires no manual developer involvement. It is therefore usable also by developers that have no prior knowledge on the system. Moreover, no preparation steps are required for effectively operating our tool. Therefore, this tool can be seamlessly integrated in modern IDEs or source control systems, thus assuring continuous validation of the software architecture, as it is being developed.

Contribution: in this paper we present an automatic solution for continuous software architecture validation. We describe the patterns used to detect architectural problems and present the tool that supports their execution on code. We validate our approach on four open source projects and one proprietary system.

The research leading to these results has received funding from the European Community’s Seventh Framework Programme (FP7/2007-2013) under grant agreement no. 610802

II. EXAMPLES

We start by showing two examples for design issues that can be detected by our tool. We will then refer back to these examples when describing the technical details of the approach in Section III. We utilize two open source projects – *org.eclipse.jdt.core* [11] and *Apache Ant* [2], and demonstrate deterioration in software architecture that could have been detected by our approach. In each example, we show different versions, where in the later versions an architectural issue has been introduced over the earlier ones. We describe what kinds of pattern analysis in our solution could have been used to identify these as they were introduced, thus helping the developers fix them when they were still “fresh”, and the cost of such a fix would have been minimal.

A. Ant

In this section we show how one architectural pattern applied to several versions of *Ant* [2] open source library exposed an underlying design issue.

Cyclic dependencies among packages (and components in general) are considered an anti-pattern that should be avoided [15]. We demonstrate how cyclic dependencies in the *Ant* project, which is implemented in Java, have become worse (i.e., larger and more tightly-coupled cycles were generated) between different versions of the software. Such deterioration could have been avoided if a continuous feedback tool had been deployed, which would have warned the developer of this situation at development time.

We start with version 1.8.0 of *Ant*. We applied automatic detection of cycles among packages to this version, and detected four cycles. Figure 1 shows three of those cycles of packages. The forth cycle consists of 23 packages and is not shown for space reasons. Note that our solution detects strongly connected components (SCCs) [39] of nodes in the graph representing the packages. Each SCC corresponds to a cyclic dependency among them.

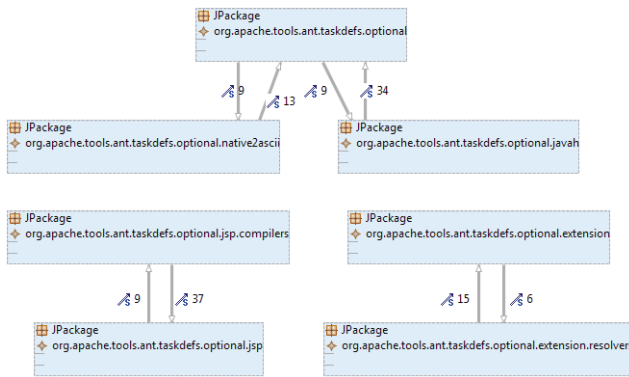


Fig. 1. Three cyclic dependencies between packages in *ant* 1.8.0. Nodes represent packages, while edges denote dependencies and are annotated with the number of such dependencies.

In version 1.9.0, two of the cycles were merged into one cycle, to which additional packages were added as well. In total, there were 29 packages in this cycle.

Being integrated with the Eclipse development environment, the tool also supports a drill down from the graph down to the underlying root-cause code fragments. For example, in order to understand why the architectural issue got worse, we looked at the new dependency in code that caused the union of the cycles. Figure 2 shows a dependency of the package *org.apache.tools.ant.taskdefs* on *org.apache.tools.ant.taskdefs.optional* that exists in version 1.9.0 but not in the earlier 1.8.0 version. This dependency exists due to a dependency of the class *XSLTProcess* (from *org.apache.tools.ant.taskdefs* package) on *TraXLiason* (from *org.apache.tools.ant.taskdefs.optional* package), as shown in the middle of the figure. This dependency, in turn, is caused by an invocation of the constructor of *TraXLiason* in the method *resolveProcessor* from *XSLTProcess*.

The implementation of *resolveProcessor* in versions 1.8.0 and 1.9.0 is given in Figure 3. On the left hand side the implementation of 1.8.0 does not contain a direct dependency on the class *TraXLiason*. This version of the code simply uses the name of the class *TraXLiason* (in line 4) and then creates an instance of the class via reflection (in lines 9 and 10). This changed in version 1.9.0, where the class is loaded via its *java.lang.Class* reference.

Both versions of the method suffer from bad design since the best solution for creating different processors would have been using a Factory pattern [16]. The process of comparing two version of the code by analyzing cyclic dependencies between high level entities (such as packages) revealed an underlying architectural problem, as the specific dependency between *org.apache.tools.ant.taskdefs* and *org.apache.tools.ant.taskdefs.optional* might not have received a special attention in any given version when analyzed separately.

B. JDT

Factory pattern [16] implementation is often violated even if the code was originally implemented correctly following this pattern. Figure 4 shows a class diagram of four classes and interfaces from *org.eclipse.jdt.core* project [11] participating in such a pattern. In this example, the *IProblemFactory* interface serves as an abstract factory, while *IProblem* acts as an interface for clients to access the generated objects. The class *DefaultProblem* is one specific implementation of the *IProblem* interface. Finally, the *Compiler* class plays the role of the client accessing the entire pattern. There is an obvious violation of the factory pattern as the client class *Compiler* is accessing the specific *DefaultProblem* implementation. In a correct implementation of the Factory design pattern, the *Compiler* would have used *IProblemFactory* in order to create an object implementing *IProblem*, and then work with that object only via this interface, without ever referring to the concrete implementing class.

Figure 5 shows a code snippet from the *handleInternalException* method in the *Compiler*

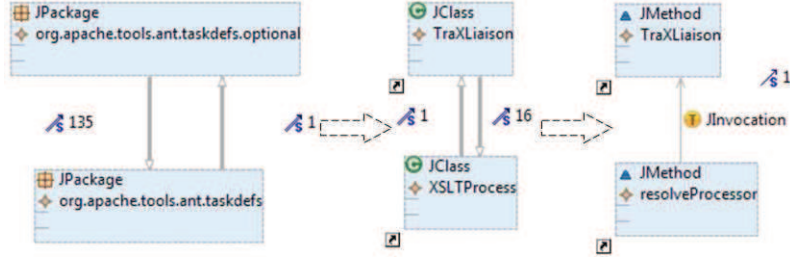


Fig. 2. Drill down of problematic new dependency in *ant* 1.9.0.

<pre> 1 private void resolveProcessor(String proc) throws Exception { 2 String classname; 3 if (proc.equals(PROCESSOR_TRAX)) { 4 classname = TRAX_LIAISON_CLASS; 5 } else { 6 //anything else is a classname 7 classname = proc; 8 } 9 Class clazz = loadClass(classname); 10 liaison = (XSLTLiaison) clazz.newInstance(); 11 } </pre>	<pre> private void resolveProcessor(String proc) throws Exception { if (proc.equals(PROCESSOR_TRAX)) { liaison = new org.apache.tools.ant.taskdefs.optional.TraXLiaison(); } else { //anything else is a classname Class clazz = loadClass(proc); liaison = (XSLTLiaison) clazz.newInstance(); } } </pre>
---	---

Fig. 3. *resolveProcessor* method implementation in versions 1.8.0 and 1.9.0.

class. The snippet was taken from two different versions of *org.eclipse.jdt.core*. On the left hand side, version 2.0.1 appears and contains no reference to *DefaultProblem* class. On the right hand side, version 3.0.1 appears and contains the problematic reference.

This violation could also be intercepted as will be explained in Section III-B3 below. Much like the previous example, by applying this analysis continuously to the code as it is being developed we could have raised a “red flag” very early in the development cycle, as soon as the violation is introduced, thus helping to avoid the issue before the code is released.

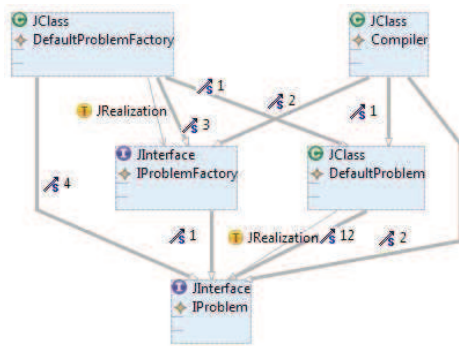


Fig. 4. Factory pattern violation in *jdt core* 3.0.1.

III. SOLUTION

Our solution is integrated within a development environment, thus providing traceability between the architectural view and the originating code. This integration also facilitates

potential inclusion of the tool as part of a nightly build process. It is comprised of two main subcomponents:

- Parsing of the source code of a given software system and generation of a generic model that represents the code. This step includes computation of high level dependencies such as those between packages, folders, jar files, projects, etc.
- Execution of a collection of configurable analytical engines that automatically detect potential architectural violations. Such potential violations are flagged by the tool and presented to the user for further manual inspection.

In this section we present the framework that we used in order to support this process, and also present the patterns that we used for evaluation of our approach.

A. Framework

The implementation of our solution is based on an existing System Grokking technology [18], [19]. This proprietary technology supports software architects in the incremental and iterative user-driven understanding of software systems through higher levels of abstraction. It is implemented as a collection of plugins for Eclipse and includes a visual modeling framework, an extension of the Eclipse Modeling Framework [14]. It can be executed on a given software artifact by simply pressing the relevant button from Eclipse menus. It is not yet integrated as part of nightly build scripts, which remains part of future work. However, [13] gives us confidence that such integration can be implemented.

The System Grokking technology is illustrated in Figure 6. This technology first transforms a given software system that the user wishes to analyze into a semantic model, via domain-specific transformations. Semantic models provide a common

```

if (result != null && !result.hasBeenAccepted) {
    /* distant problem which could not be reported back there */
    if (abortException.problemId != 0) {
        result
        .record(
            problemReporter
            .createProblem(result.getFileName(),
                abortException.problemId,
                abortException.problemArguments,
                Error, // severity
                0, // source start
                0, // source end
                0, // line number
                unit, result),
            unit);
    } else {
        /* distant internal exception which could not be reported back there */
    }
}

```

```

if (result != null && !result.hasBeenAccepted) {
    /* distant problem which could not be reported back there? */
    if (abortException.problem != null) {
        recordDistantProblem: {
            IProblem distantProblem = abortException.problem;
            IProblem[] knownProblems = result.problems;
            for (int i = 0; i < result.problemCount; i++) {
                if (knownProblems[i] == distantProblem) // already recorded
                    break recordDistantProblem;
            }
            if (distantProblem instanceof DefaultProblem) { // fixup filename TODO
                ((DefaultProblem) distantProblem).setOriginatingFileName(result.get
            }
            result .record(distantProblem, unit);
        }
    } else {
        /* distant internal exception which could not be reported back there */
    }
}

```

Fig. 5. Code snippet from *handleInternalException* implementation in jdt core, versions 2.0.1 (left) and 3.0.1 (right).

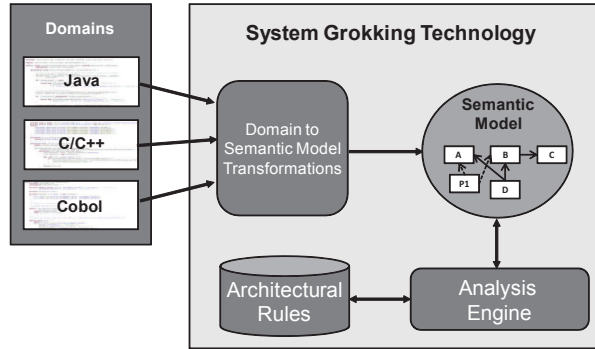


Fig. 6. System Grokking technology overview.

representation of the information required for analysis on top of different domains, and are also used for visualization and persistency of results. Semantic models can represent various software elements such as packages, classes, functions and variables. These elements are represented as nodes with fully qualified names and types. The models also represent relationships among those elements. Relationships are directed edges with weight and type. For example, two elements representing two functions may have a relationship of type *Invocation* and weight 2 if one function calls the other two times. Another example was given in Figure 1, where the seven nodes depict the packages, and the edges depict dependencies between packages. The edges are annotated with the corresponding weights assigned to them.

Once the software artifact is parsed via a domain specific parser, and converted into a persistable domain agnostic model – the model can be analyzed using various architectural patterns. The configuration and the results of the patterns analysis are persisted inside the model, thus allowing the users to investigate the results at their convenience. The architectural validation patterns presented in this paper are implemented in the System Grokking technology.

Although in this paper we focused on Java, the solution is not limited to this domain, as the analysis performed by the tool is domain agnostic. Currently, our framework supports Java, C/C++ and Cobol languages.

B. Patterns

Our tool can automatically apply a list of predefined patterns and supports definition of new patterns by the user. In Section II we demonstrated how detection of cyclic dependencies and violation of factory patterns helped detect problems. In this section we will elaborate on these and other patterns.

1) *Cyclic Dependency Detection*: Automatic detection of cyclic dependencies can be performed at various levels of granularity. For large scale software systems the tool may be used to detect patterns between projects. For medium size applications it can be done at the package level. While class level and method level analyses are also possible, we found them to be less useful.

Dependencies among packages and projects are computed based on dependencies among lower level elements, such as classes, that they contain. Specifically, two projects depend on each other only if at least one package from one of the projects depends on at least one package from the other project. Packages depend on each other only if at least one class from one of the packages depends on at least one class from the other packages. Classes depend on each if they, for example, extend each other or contain methods that call methods from the other class.

Detection of cycles is performed using Tarjan's [39] algorithm for detecting strongly connected components in graphs. Our tool uses packages (or projects) as nodes in its graph and the dependencies among packages as edges. The result of the analysis is a group of strongly connected components representing the different cycles. The result is persistable. This allows computing cycles for various versions of the code and comparing the subgraphs representing those cycles. Whenever two cycles are merged or a new cycle emerges, the tool can highlight the dependencies causing the problematic change in the architecture. The user can then drill down to the code to view the actual change in the code causing the issue.

2) *Layering Violations Detection*: Automatic computation of layers and their violations detection is normally performed on packages. However, it can be performed on *jar* archives or projects just as cycles detection. Layering is performed bottom-up. Our tool first looks for packages that have no dependencies on other packages (utility packages, for example) and places them in the bottom most layer, *L1*. It then looks for all the packages that have dependencies on packages from

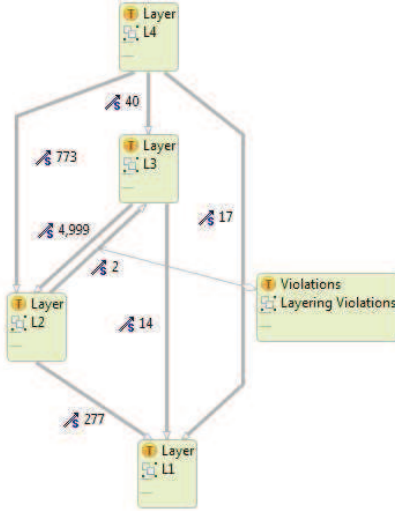


Fig. 7. Automatic layering of packages in *ant* 1.8.0.

L1 but not on any other packages and place them in layer *L2*. This process continues until all the packages are assigned to some layer.

To handle cycles and detect violations, the tool first goes over all pairs of packages that have cyclic dependency on each other. It compares the weights that these dependencies have. If the ratio between the weights is smaller than some predefined threshold, the tool ignores the dependency with the lower weight. Any remaining cycle in the graph is then treated as a single node.

The result of the automatic layering computed for *ant* 1.8.0 is shown in Figure 7. Four layers have been detected, denoted *L1*, *L2*, *L3*, *L4*, with a violation of this layering detected between layer *L2* and *L3* – two dependencies exist between packages in *L2* and those in *L3*. Compared to almost five-thousand dependencies in the other direction, the tool detected this as a violation of intended layering, as opposed to merging the layers into a single larger layer.

Automatic layering can also be performed on manually defined components. In this case, the user may manually define logical components in the system by grouping multiple packages or classes together. The tool will then compute dependencies between those components based on the elements that these components contain. It will consequently compute the layering on these components and look for violations.

3) *Factory Pattern Detection*: This violation is intercepted by applying a generic pattern recognition over the abstract graph representation of the system. Specifically, the relevant pattern does the following: a) finds factory classes and interfaces *FC* and *FI*, e.g. by names ending with **Factory* and the relevant semantic types (class or interface); b) finds classes *C* that refer to factory interfaces from *FI*; c) detects interfaces *Int* that factory interfaces from *FI* refer to and implementations *Impl* that factory classes from *FC* depend on; and d) verifies that classes in *C* only refer to interfaces in *Int*.

A dependency of a class in *C* directly on an implementation in *Impl* constitutes a violation of the factory pattern, as in Figure 4.

4) *User Defined Pattern Detection*: Our generic and extendable framework supports implementation of user defined patterns at two different levels. The first is programmatical extension of an Eclipse plugin extension point. We used this option to implement support for Factory pattern detection. Another example of such extensions would be detection of superfluous file inclusions. In this case, one could detect header files (in C language) that are included in a source file, while none of their contents is used in the source file. This often happens when some declarations move from one header file to another and the header files are simply included in one another. Removal of unused *include* directives may improve build and test time. Therefore we flag such cases as possible violations.

Another option for user defined pattern specification is to create and execute a query by drawing the pattern elements and their dependencies in our visualization framework. An example of such “Query by Example” is shown in Figure 8(a). The query is used for detecting header files (in C language) that depend on source files. The results of the execution of such query are shown in Figure 8(b). They were acquired when analyzing on *grub* boot loader package [17], version 1.97.2.

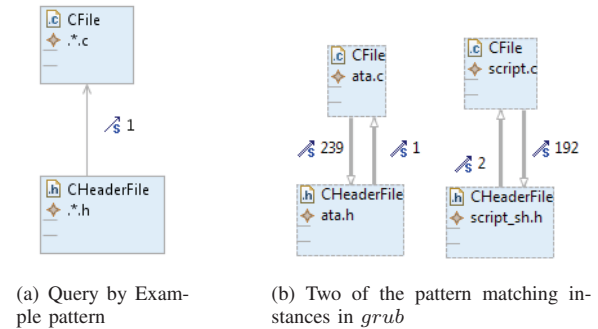


Fig. 8. Query by Example pattern definition and results

IV. USECASES

We applied our solution to a tool developed within our organization. The direct connection with the developers of this tool allowed us to get feedback on all violations detected by our solution and, eventually, see the architecture of the tool changing and getting closer to its original and intended design. We also performed analysis of relatively large open source applications, Weka [3] and Vuze [40], in order to check the scalability of our solution.

We performed the analysis on multiple versions of each artifact and saw that in many cases analysis of two consequent minor versions, such as *ant*1.7.1 versus *ant*1.7.0 either lead us to no interesting insights or to too many violations. The former case occurred when there were too few architectural changes between the versions. The later case occurred when there were changes that caused a lot of architectural violations.

However, in a consequent major release some of the violations were fixed. We believe this is due to the nature of major release having significant improvements, accompanied by refactorings and better code reviews. We therefore chose to present pairs of versions that contained only a small number of violations, as these are easier to follow, but cover the same types of architectural violations.

A. An Internally Developed Tool

We used the layering pattern and the cyclic dependencies pattern in order to detect architectural violations in an internally developed tool. The tool is a Java application with about 70K lines of Java code organized within 700 classes, in 37 packages. The entire process of code parsing, model creation and patterns detection took less than half a minute on a Lenovo laptop with 8GB of memory and i7 processor with 2.2GHz clock speed.

We considered two consecutive releases of the tool at first. We will refer to them as *ver1* and *ver2* further on. The layering detected by our tool is shown in Figure 9. On the left hand side, the layering of the packages in *ver1* is presented whereas on the right hand side, the layering of the packages in *ver2* appears. At first sight, the layering in *ver2* may seem to be an improved version of that in *ver1*. However, a more thorough investigation shows that layers *L3* and *L4* in *ver1* were merged into one single layer *L3* in *ver2*. This happened because a violating dependency *d*, that appears on the left hand side as a dependency of *L2* on *L3* of weight 6 got stronger, as shown in Figure 10. The layering threshold was set to 10%, such that when our tool compared 6 dependencies against 89, it put the packages with these dependencies in two different layers. When the ratio changed to 10 versus 74, it went above this threshold. Therefore, the automatic layering algorithm put all the packages from *L2* and *L3* in the same layer in *ver2*.

Figure 10 shows two packages from the tool, with obfuscated names. In *ver1* there were 89 dependencies of package *com.obfuscated.p1* on package *com.obfuscated.p2* whereas in the opposite direction there were only six. By further drilling down into the dependency with weight 6, as shown in Figure 11, we learned that class *B* from *com.obfuscated.p2* had 6 dependencies on class *A* from *com.obfuscated.p1*, which caused the problematic dependency among the packages. When we looked at the same packages in *ver2* we

learned that the same 6 dependencies of class *B* on class *A* remained, while there were 4 new dependencies of class *C* from *com.obfuscated.p2* on class *A*. This means that the architectural problem that existed in *ver1* got worse in *ver2* as more dependencies on class *A* from classes in package *com.obfuscated.p2* evolved.

We brought this to the attention of the developers of the tool who decided that some refactoring of the code was necessary in order to resolve this. Once a new version (*ver3*) became available, we ran our analysis again. This time we found no problematic dependencies between the two packages.

In parallel, the developers of the tool performed refactorings to resolve multiple cyclic dependencies. Our tool showed that there were three cycles of packages in the system. The smallest cycle contained only two packages with the other two cycles containing 6 and 11 packages respectively. Two of the cycles were relatively simple to resolve by some refactorings. The third cycle detected by the tool stemmed from a very large number of dependencies among different UI classes and data classes. One possible solution would be to put all of these classes in the same package, which would clearly violate the original design. This case required more thorough manual restructuring and was not handled in *ver3*. The layering of packages in *ver3* brings this problem to light in Figure 12 by marking 55 dependencies of layer *L2* (containing the aforementioned data classes) on *L5* (containing the UI classes) as violating ones.

Note that the user can choose which violations to fix and which to ignore or delay resolution for. As in other development support tools, our solution simply flags the issues for the developer's attention.

B. Large Open Source Application

To learn how scalable and practically applicable our solution is, we tried our tool on two large open source applications. First we analyzed two version of *Weka* [3] which is a 250K lines of code data mining software application. We compared two versions: 3.4 and 3.5.8. We investigated the results of the layering pattern and learned that even at this scale it was easy to understand the violations. Specifically, as shown in Figure 13, there was a direct cyclic dependency in version 3.5.8 of *weka.core* on *weka.gui* package that did not exist in version 3.4. Although we have no knowledge on the original

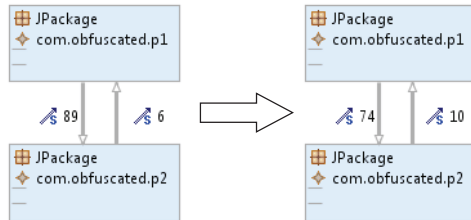


Fig. 10. Cyclic dependency between two packages in *ver1* (on the left) and *ver2* (on the right)

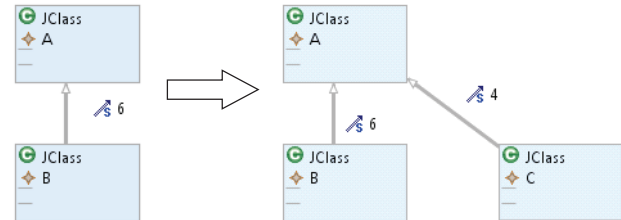


Fig. 11. Problematic dependencies between classes of two packages in *ver1* (on the left) and *ver2* (on the right)

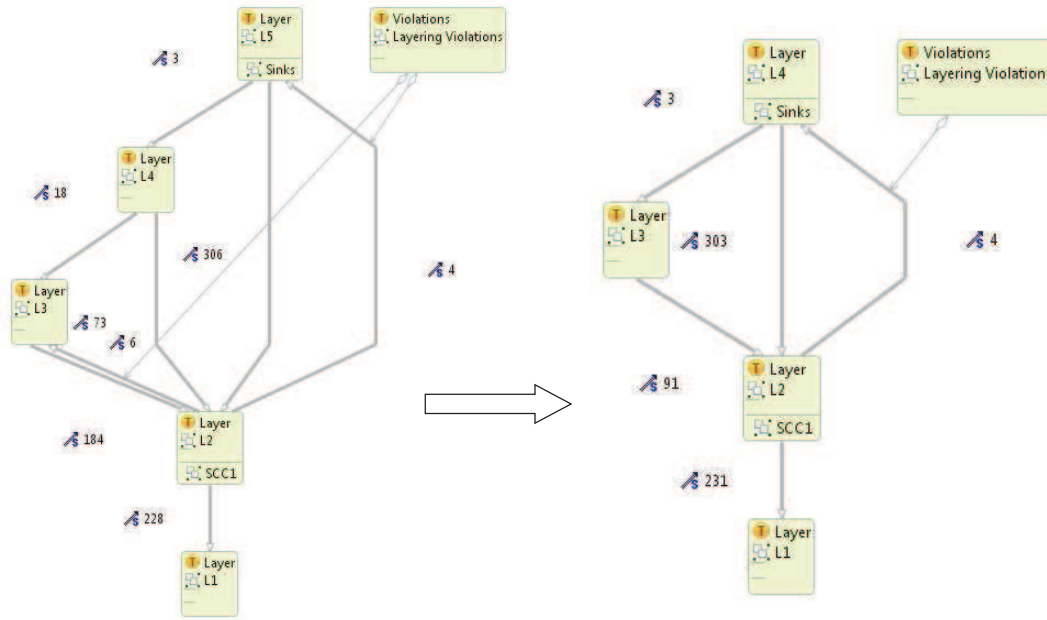


Fig. 9. Layering of packages in two consecutive versions of the code.

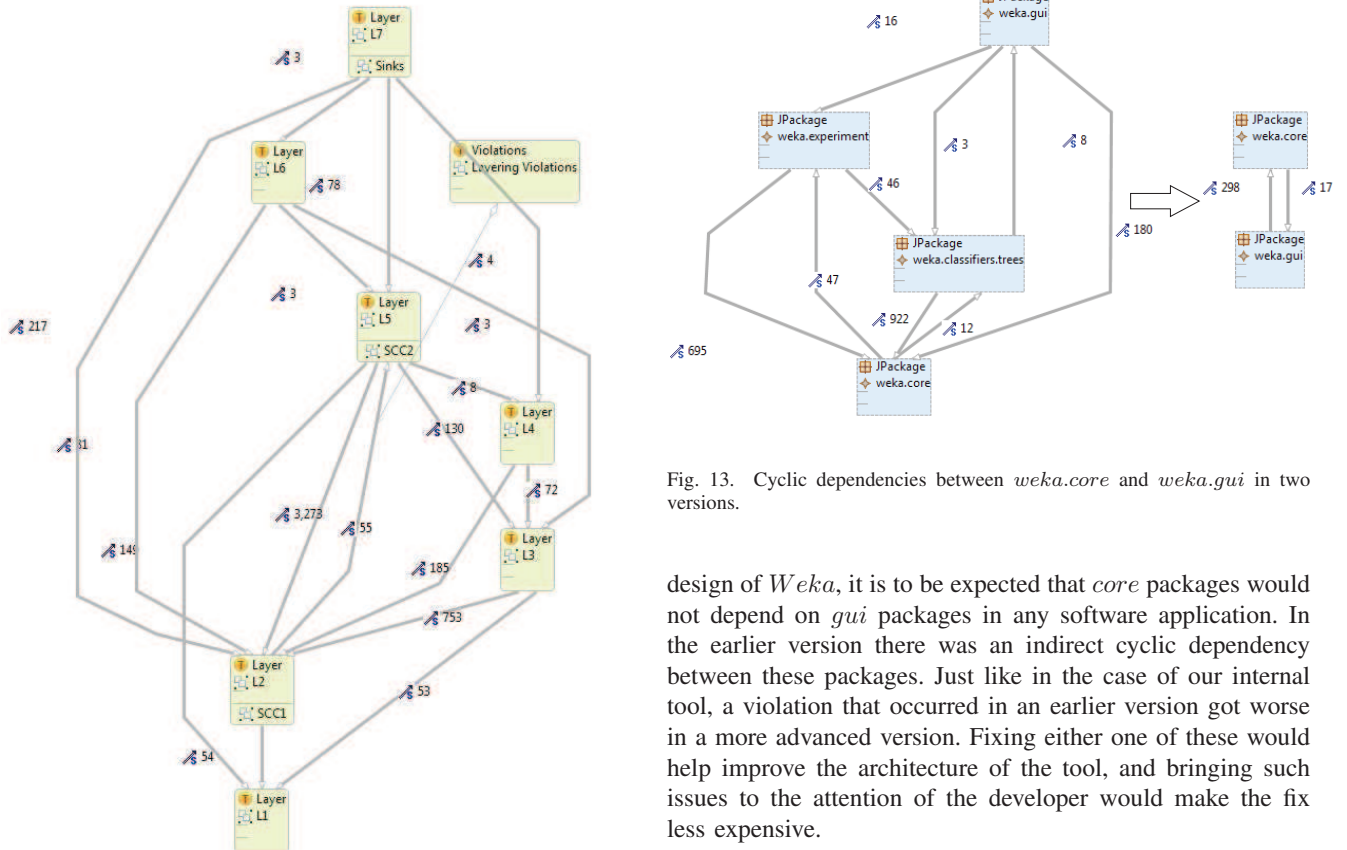


Fig. 12. Layering of packages after code fixes.

Fig. 13. Cyclic dependencies between *weka.core* and *weka.gui* in two versions.

design of *Weka*, it is to be expected that *core* packages would not depend on *gui* packages in any software application. In the earlier version there was an indirect cyclic dependency between these packages. Just like in the case of our internal tool, a violation that occurred in an earlier version got worse in a more advanced version. Fixing either one of these would help improve the architecture of the tool, and bringing such issues to the attention of the developer would make the fix less expensive.

We then analyzed *Vuze* [40], a bit torrent client containing half a million lines of code. The most interesting insights for this case came from the cycles detection pattern which showed

that as the application evolved more and more packages become cyclically dependant on each other.

The scalability and applicability of our solution can be learned from Table I. It shows three artifacts described in this section of sizes varying from 70K lines of code to 500K. The forth column of the table shows the total processing time, while the last column shows the time that was consumed during patterns execution, both in seconds. In all three cases we applied cycles and layering detection. All of the executions were on the same machine (*i7* processor, 2.2GHz, 8GB of memory).

The table shows that most of the time our tool spends on parsing the code, creating the generic model representing the code and saving the model on disk. The actual patterns detection takes less than 16% of the total time. It also shows that the analysis can only be performed on demand or during nightly builds when the developers expect to have longer processing time. Finally, it shows that the solution is scalable as the processing of an application with half a million lines of code took less than three minutes. Moreover, addition of new patterns will only increase the patterns processing time and will not change the code analysis time.

TABLE I
EXECUTION TIME IN SECONDS FOR DETECTING CYCLIC DEPENDENCIES
AND LAYERING IN DIFFERENT ARTIFACTS.

	KLOC	#Packages	Total Processing Time (sec)	Patterns Processing Time (sec)
Internal tool	70	37	23	1
Weka	250	285	31	2
Vuze	500	473	154	24

V. RELATED WORK

Eichberg et. al. introduced continuous validation of software architecture into the software development process in [13]. Unlike our approach, in that work the developer is required to manually define the components (termed ensembles) of the system, and the required constraints among them. A domain-specific language is introduced for specification of the ensembles and the constraints on their dependencies. Using this language, the developer can state explicit constraints such as required layering structure, or Factory design pattern relationships. However, as opposed to our work, this specification needs to be supplied explicitly, stating the concrete artifacts that participate in the requirement, while in our work the requirement is to be stated as a general pattern to be enforced throughout the system.

Microsoft introduced a feature into its Visual Studio IDE that allows the developer to check conformance of code with a given layering diagram [33]. Unlike our solution, the layering diagram needs to be manually defined in order for the system to check consistency. Moreover, it is limited to this one kind of analysis of the code and architecture as opposed to our framework that allows to easily introduce new patterns that yield new analyses. Other commercial and open source tools

evaluate the quality of a system by either computing various metrics on the code [37], detecting errors in the code itself [7], [29] or validating the attendended architecture of the code [25].

Other methods for ensuring high conformance of implementation with an architecture explicitly specified in some format are presented, e.g. in [8], [24], [44]. CodeCharts, a language for manually defining architectural decisions and requirements, enabling a validation process of the program against this model, is introduced in [34].

An approach for continuous validation of object-oriented best practices is presented in [21]. This approach checks for low-level code violations of standard best practices of object-oriented code, such as class size and the size of inheritance trees. Similar functionality is supplied by the Eclipse plugin EclipseMetrics [30].

Andreas Grabner [20] developed a dynaTrace framework that allows to use existing unit tests for architectural rules validation. However, his notion of architectural rules is different from ours and represents requirements from the system implementation and not how it is implemented.

Bouwers et. al. [5], [6] developed metrics for quantifying the level of analyzability of systems, and the level of encapsulation and independence within a system. In both cases, given a system, a single metric is computed that is aimed at capturing a certain property of the system. While useful, the purpose and usage of such a metric is different from the purpose of our analysis – by supplying a single numeric metric, the developer or architect may get a “grade” for a certain aspect of the system, thus supporting evaluation of systems. The purpose of our analysis is to aid the developer in the course of system development and evolution.

The Arcon tool [12] automatically validates UML models at different levels – that a system model conforms to an architectural meta-model. Similarly, [27] proposes a system allowing to define validation rules for ensuring conformance of architecture with meta-models.

In [31], an exploratory analysis of automatically-generated code anomalies is performed, for the purpose of studying whether automatically-detected code anomalies may be sufficient for identifying architectural issues. Their conclusion was negative – many of the code anomalies detected were not related to architectural issues, and over 50% of the anomalies not identified by the employed techniques were actually correlated to architectural problems.

One approach complementary to ours is that of introduction of architecture description languages (ADLs). These are formal languages in which one may formally model software architecture, and then formally reason about the resulting architecture. For example, in [41], an architecture description language (ADL) is introduced, focused on supporting architecture-based evolution. The approach presented in [32] tries to bridge the gap between formal and informal languages for architecture description. Once captured in a formal language, one can perform model checking on the architectural design in order to verify compliance of the architecture with

different properties [43]. A method that allows extension of ADLs is presented in [9].

VI. CONCLUSION AND FUTURE WORK

In this work, we presented an automated approach for detecting architectural problems. Our solution provides insights on large systems, and requires no prior knowledge on the system from the developers. Moreover, no preparation steps are required for effectively operating our tool. The tool has been evaluated on several open source projects, where it shows promising results on issues that could have been detected, had the tool been applied during development. Moreover, we have leveraged the tool during development of an internally developed tool, where the automatic architecture validation approach has introduced significant improvement in the architecture of the software.

The technology relies on the System Grokking technology, a general framework for high-level analysis of software systems. In this framework, the system is first (automatically) transferred into a high-level abstract graph-based representation. The graph captures components at different levels (e.g. packages), and their dependencies. Graph analysis algorithms can then be applied to automatically detect and highlight problematic patterns as they evolve. Two examples of such patterns are cyclic dependencies and violations of design patterns such as the Factory pattern.

While the tool is implemented and integrated with the Eclipse IDE, it is currently invoked manually, as opposed to being automatically triggered by the build system. In the future, we plan to integrate it with an automatic build system, such as Maven or Ant, in order to achieve true continuous validation, e.g. in every nightly build of a system. Additional feature would allow the user to mark violations as those that are acceptable and should not be flagged with every execution of the tool. The tool would allow the user to ignore some violations for a specified period of time, in case the user plans some significant refactoring effort. It may also allow the user to ignore some dependencies indefinitely.

Another direction of future work is that of leveraging formal methods for software architecture, such as architecture description languages (ADLs). Given a formal description of the intended architecture, our tool can automatically extract the actual software architecture and try to map between the two. Any discrepancies will then be reported to the developer.

REFERENCES

- [1] Architecture-based approach to reliability assessment of software systems. *Perform. Eval.*, 45(2-3):179–204, July 2001.
- [2] Apache. The Apache Ant Project, 2014.
- [3] Machine Learning Group at the University of Waikato. Weka 3: Data mining software in java, 2014.
- [4] Sami Beydeda, Matthias Book, Volker Gruhn, et al. *Model-driven software development*, volume 15. Springer, 2005.
- [5] Eric Bouwers, José Pedro Correia, Arie van Deursen, and Joost Visser. Quantifying the analyzability of software architectures. In *9th Working IEEE/IFIP Conference on Software Architecture, WICSA 2011, Boulder, Colorado, USA, June 20-24, 2011*, pages 83–92, 2011.
- [6] Eric Bouwers, Arie van Deursen, and Joost Visser. Dependency profiles for software architecture evaluations. In *IEEE 27th International Conference on Software Maintenance, ICSM 2011, Williamsburg, VA, USA, September 25-30, 2011*, pages 540–543, 2011.
- [7] Coverity. Coverity, 2014.
- [8] Florian Deissenboeck, Lars Heinemann, Benjamin Hummel, and Elmar Juergens. Flexible architecture conformance assessment with conqat. In *Proceedings of the 32Nd ACM/IEEE International Conference on Software Engineering - Volume 2, ICSE '10*, pages 247–250, New York, NY, USA, 2010. ACM.
- [9] Davide Di Ruscio, Ivano Malavolta, Henry Muccini, Patrizio Pelliccione, and Alfonso Pierantonio. Developing next generation adls through mde techniques. In *Proceedings of the 32Nd ACM/IEEE International Conference on Software Engineering - Volume 1, ICSE '10*, pages 85–94, New York, NY, USA, 2010. ACM.
- [10] Paul Duvall, Stephen M. Matyas, and Andrew Glover. *Continuous Integration: Improving Software Quality and Reducing Risk (The Addison-Wesley Signature Series)*. Addison-Wesley Professional, 2007.
- [11] Eclipse. Eclipse Java Development Tools (JDT), 2014.
- [12] Eclipselab. Architectural Conformance Validation Tool for MDD, 2014.
- [13] Michael Eichberg, Sven Kloppenburg, Karl Klose, and Mira Mezini. Defining and continuous checking of structural program dependencies. In *Proceedings of the 30th International Conference on Software Engineering, ICSE '08*, pages 391–400, New York, NY, USA, 2008. ACM.
- [14] EMF. Eclipse modeling framework. <http://www.eclipse.org/modeling/emf/>, 2012.
- [15] J.-R. Falleri, S. Denier, J. Laval, P. Vismara, and S. Ducasse. Efficient retrieval and ranking of undesired package cycles in large software systems. In *Proceedings of the 49th international conference on Objects, models, components, patterns, TOOLS'11*, pages 260–275. Springer-Verlag, 2011.
- [16] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-oriented Software*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 1995.
- [17] GNU. Gnu grub, 2014.
- [18] M. Goldstein and D. Moshkovich. System grokking: a novel approach for software understanding, validation, and evolution. In *Proceedings of the 7th international conference on Next Generation Information Technologies and Systems, NGITS'09*, pages 38–49. Springer-Verlag, 2009.
- [19] Maayan Goldstein and Dany Moshkovich. Improving software through automatic untangling of cyclic dependencies. In *Proceedings of the 36th International Conference on Software Engineering, ICSE '14, Companion Proceedings, Hyderabad, India, May 31 - June 07, 2014*, pages 155–164, 2014.
- [20] Andreas Grabner. Performance and architecture validation with existing unit tests, 2014.
- [21] Maen Hammad, Mustafa Hammad, and Mohammad Bsoul. An approach to automatically enforce object-oriented constraints. *Int. J. Comput. Appl. Technol.*, 49(1):50–59, February 2014.
- [22] N. Harrison and P. Avgeriou. Pattern-based architecture reviews. *Software, IEEE*, 28(6):66–71, Nov 2011.
- [23] David Hovemeyer and William Pugh. Finding Bugs is Easy. *SIGPLAN Not.*, 39(12):92–106, December 2004.
- [24] S. Huynh, Yuanfang Cai, Yuanyuan Song, and K. Sullivan. Automatic modularity conformance checking. In *Proceedings of the 30th International Conference on Software Engineering*, pages 411–420, May 2008.
- [25] IBM. Rational software architect simulation toolkit, 2014.
- [26] David S Janzen and Hossein Saiedian. Test-driven development: Concepts, taxonomy, and future direction. *Computer Science and Software Engineering*, page 33, 2005.
- [27] Mohamed Hadj Kacem, Mohamed Jmaiel, Ahmed Hadj Kacem, and Khalil Drira. An uml-based approach for validation of software architecture descriptions. In *Proceedings of the 2Nd International Conference on Trends in Enterprise Application Architecture, TEAA'06*, pages 158–171, Berlin, Heidelberg, 2007. Springer-Verlag.
- [28] Rick Kazman, Leonard J. Bass, Mike Webb, and Gregory D. Abowd. Saam: A method for analyzing the properties of software architectures. In *Proceedings of the 16th International Conference on Software Engineering, Sorrento, Italy, May 16-21, 1994*, pages 81–90, 1994.
- [29] Klockwork. Rogue wave software, 2014.
- [30] Lance Walton and Channing Walton. EclipseMetrics. <http://www.stateofflow.com/projects/16/eclipsemetrics>, 2005.

- [31] Isela Macia, Joshua Garcia, Daniel Popescu, Alessandro Garcia, Nenad Medvidovic, and Arndt von Staa. Are automatically-detected code anomalies relevant to architectural modularity?: An exploratory analysis of evolving systems. In *Proceedings of the 11th Annual International Conference on Aspect-oriented Software Development*, AOSD '12, pages 167–178, New York, NY, USA, 2012. ACM.
- [32] N. Medvidovic, D.S. Rosenblum, and R.N. Taylor. A language and environment for architecture-based software development and evolution. In *Software Engineering, 1999. Proceedings of the 1999 International Conference on*, pages 44–53, May 1999.
- [33] Microsoft. Validate code with layer diagrams, 2014.
- [34] Jon Nicholson, Amnon H. Eden, Epameinondas Gasparis, and Rick Kazman. Automated verification of design patterns: A case study. *Sci. Comput. Program.*, 80:211–222, February 2014.
- [35] Mary Shaw and Paul Clements. The golden age of software architecture. *IEEE Softw.*, 23(2):31–39, March 2006.
- [36] Mary Shaw, Robert DeLine, Daniel V. Klein, Theodore L. Ross, David M. Young, and Gregory Zelesnik. Abstractions for software architecture and tools to support them. *IEEE Trans. Softw. Eng.*, 21(4):314–335, April 1995.
- [37] SonarQube. Sonarqube software, 2014.
- [38] K. J. Sullivan, W. Griswold, Y. Cai, and B. Hallen. The structure and value of modularity in software design. In *Proceedings of the 8th European software engineering conference held jointly with 9th ACM SIGSOFT international symposium on Foundations of software engineering*, ESEC/FSE-9, pages 99–108. ACM, 2001.
- [39] R. E. Tarjan. Depth-first search and linear graph algorithms. *SIAM J. Comput.*, 1(2):146–160, 1972.
- [40] Vuze. Vuze bittorrent client, 2014.
- [41] Rainer Weinreich and Georg Buchgeher. Paving the road for formally defined architecture description in software development. In *Proceedings of the 2010 ACM Symposium on Applied Computing*, SAC '10, pages 2337–2343, New York, NY, USA, 2010. ACM.
- [42] S. Wong, Y. Cai, M. Kim, and M. Dalton. Detecting software modularity violations. In *Proceedings of the 33rd International Conference on Software Engineering*, ICSE '11, pages 411–420. ACM, 2011.
- [43] Pengcheng Zhang, Henry Muccini, and Bixin Li. A classification and comparison of model checking software architecture techniques. *J. Syst. Softw.*, 83(5):723–744, May 2010.
- [44] Yongjie Zheng and Richard N. Taylor. Enhancing architecture-implementation conformance with change management and support for behavioral mapping. In *Proceedings of the 34th International Conference on Software Engineering*, ICSE '12, pages 628–638, Piscataway, NJ, USA, 2012. IEEE Press.